# Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

**AY: 2026-27**

| | | | |
|---|---|---|---|
| **Class:** | **TE-AI&DS** | **Semester:** | **V** |
| **Course Code:** | **2015111** | **Course Name:** | **Statistics for Machine Learning and Data Science Lab** |

| | |
|---|---|
| **Name of Student:** | **Anish M. Naik** |
| **Roll No. :** | **79** |
| **Experiment No.:** | **3** |
| **Title of the Experiment:** | **Implement Correlation Analysis, Distance Measures, and Data Preprocessing Techniques** |
| **Date of Performance:** | |
| **Date of Submission:** | |

## Evaluation

| Performance Indicator | Max. Marks | Marks Obtained |
|---|---|---|
| Performance | 5 | |
| Understanding | 5 | |
| Journal work and timely submission | 10 | |
| Total | 20 | |

| Performance Indicator | Exceed Expectations (EE) | Meet Expectations (ME) | Below Expectations (BE) |
|---|---|---|---|
| Performance | 4-5 | 2-3 | 1 |
| Understanding | 4-5 | 2-3 | 1 |
| Journal work and timely submission | 8-10 | 5-8 | 1-4 |

**Checked by**

**Name of Faculty :**

**Signature :**

**Date :**

**Aim:** To implement correlation analysis, distance measures, and basic data preprocessing techniques on the Pima Indians Diabetes Dataset to identify relationships between variables and prepare data for further analysis.

Objectives

After completing this experiment, the student will be able to:

1. Apply correlation and distance measures to analyze relationships and similarities among data observations.
2. Implement basic data preprocessing techniques to prepare a dataset for statistical analysis and machine learning.

Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Dataset Details

Dataset: Pima Indians Diabetes Dataset

The dataset contains medical diagnostic information for 768 female patients and is used to study factors associated with diabetes.

The dataset contains numerical attributes and a binary outcome variable indicating whether the patient is diabetic.

Procedure

1. Load the Pima Indians Diabetes Dataset.
2. Identify missing or invalid values, particularly zero values in medical attributes where zero may be unrealistic.
3. Handle selected missing values using an appropriate imputation technique.
4. Calculate the correlation matrix for numerical variables.
5. Visualize correlations using a heatmap.
6. Calculate Euclidean, Manhattan, and Cosine distances for selected observations.
7. Apply an appropriate normalization or standardization technique.
8. Compare the data before and after preprocessing.
9. Interpret the obtained correlation and distance measures.

Description of the Experiment

Correlation analysis helps determine the strength and direction of relationships between variables.

For example, the relationship between:

$$Glucose \text{and} BMI$$

can be analyzed using correlation.

Distance measures determine the similarity or dissimilarity between observations.

Data preprocessing is performed to improve the quality and consistency of data before further statistical or machine learning analysis.

The experiment follows: Raw Dataset, Identify Invalid/Missing Values, Data Preprocessing, Correlation Analysis, Distance Calculation and Interpretation

Detailed Description of Statistical Methods

Correlation

Correlation measures the strength and direction of the linear relationship between two variables.

The Pearson correlation coefficient is:

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

The value of $r$ lies between:

$$-1 \leq r \leq 1$$

$r \approx 1$: Strong positive relationship

$r \approx -1$: Strong negative relationship

$r \approx 0$: Weak or no linear relationship

A correlation heatmap can be used to visualize relationships among multiple variables.

Euclidean Distance

Euclidean distance between two observations is:

$$d(\mathbf{x},\mathbf{y}) = \sqrt{\quad}(x - y)$$

It represents the straight-line distance between two points.

Manhattan Distance

Manhattan distance is:

$$d(\mathbf{x}, \mathbf{y}) = \quad | x - y |$$

It measures distance by summing the absolute differences between corresponding features.

Cosine Distance

Cosine similarity is:

$$\cos(\theta) = \frac{\mathbf{x} \cdot \mathbf{y}}{\| \mathbf{x} \| \| \mathbf{y} \|}$$

Cosine distance is:

$$d_{\text{cosine}} = 1 - \cos(\theta)$$

It measures the difference in the direction of two vectors.

Data Preprocessing

Missing Value Handling

Invalid or missing values may be replaced using techniques such as:

a. Mean imputation

b. Median imputation

For medical variables, median imputation may be preferred when extreme values are present.

Normalization

Min-Max normalization transforms values into a specified range, usually [0' 1]:

$$x = \frac{x - \min(x)}{- \min(x)} \max(x)$$

Standardization

Standardization transforms a variable to have mean 0 and standard deviation 1:

$$z = \frac{x - \mu}{\sigma}$$

Scaling is important when distance-based methods are used because variables with larger numerical ranges can otherwise dominate the distance calculation.

## Code:-

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.metrics.pairwise import (
    euclidean_distances,
    manhattan_distances,
    cosine_distances
)

df = pd.read_csv("diabetes.csv")

# Clean column names
df.columns = df.columns.str.strip().str.lower()

print("Columns in Dataset:")
print(df.columns.tolist())
```

```
print("\nFirst 5 Rows:")
print(df.head())



columns = [
    'glucose_concentration',
    'diastolic_blood_pressure',
    'triceps_sf_thickness',
    'serum_insulin',
    'bmi'
]

missing = [col for col in columns if col not in df.columns]

if missing:
    print("\nERROR!")
    print("These columns were not found:", missing)
    print("\nAvailable columns are:")
    print(df.columns.tolist())
    exit()

df[columns] = df[columns].replace(0, np.nan)

print("\nMissing Values:")
print(df.isnull().sum())



imputer = SimpleImputer(strategy="median")
df[columns] = imputer.fit_transform(df[columns])
```

```
print("\nMissing Values After Imputation:")
print(df.isnull().sum())



corr = df.corr(numeric_only=True)

print("\nCorrelation Matrix:")
print(corr)



plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()



if "outcome" in df.columns:
    sample = df.drop("outcome", axis=1).iloc[0:2]
else:
    sample = df.iloc[0:2]

print("\nSample Used for Distance:")
print(sample)

print("\nEuclidean Distance:")
print(euclidean_distances(sample))

print("\nManhattan Distance:")
```

```
print(manhattan_distances(sample))

print("\nCosine Distance:")
print(cosine_distances(sample))



if "outcome" in df.columns:
    features = df.drop("outcome", axis=1)
else:
    features = df.copy()

scaler = StandardScaler()
scaled = scaler.fit_transform(features)

scaled_df = pd.DataFrame(scaled, columns=features.columns)

print("\nStandardized Data:")
print(scaled_df.head())



scaled_sample = scaled_df.iloc[0:2]

print("\nEuclidean Distance After Scaling:")
print(euclidean_distances(scaled_sample))

print("\nManhattan Distance After Scaling:")
print(manhattan_distances(scaled_sample))

print("\nCosine Distance After Scaling:")
```

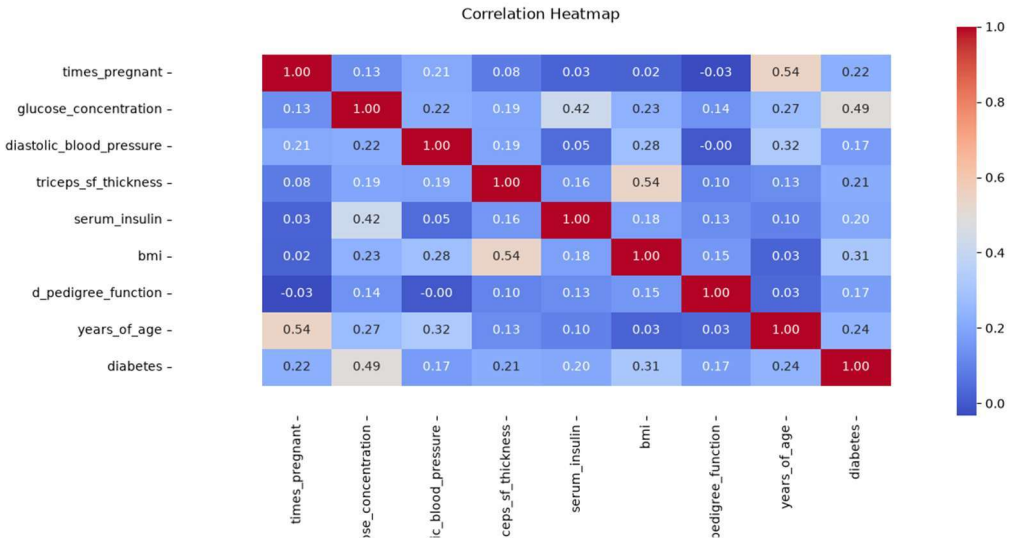
```
print(cosine_distances(scaled_sample))



if "glucose" in features.columns:
    plt.figure(figsize=(12,5))

    plt.subplot(1,2,1)
    plt.hist(features["glucose"], bins=20)
    plt.title("Original Glucose")

    plt.subplot(1,2,2)
    plt.hist(scaled_df["glucose"], bins=20)
    plt.title("Standardized Glucose")

    plt.tight_layout()
    plt.show()
```

## Output:-

```
PS D:\swayam> & c:\python313\python.exe d:/swayam/exp3.py
Columns in Dataset:
['times_pregnant', 'glucose_concentration', 'diastolic_blood_pressure', 'triceps_sf_thickness', 'serum_insulin', 'bmi', 'd_pedigree_function', 'years_of_age', 'diabetes']

First 5 Rows:
   times_pregnant  glucose_concentration  ...  years_of_age  diabetes
0               6                    148  ...            50         1
1               1                     85  ...            31         0
2               8                    183  ...            32         1
3               1                     89  ...            21         0
4               0                    137  ...            33         1

[5 rows x 9 columns]

Missing Values:
times_pregnant              0
glucose_concentration       5
diastolic_blood_pressure    35
triceps_sf_thickness        227
serum_insulin               374
bmi                         11
d_pedigree_function         0
years_of_age                0
diabetes                    0
dtype: int64

Missing Values After Imputation:
times_pregnant              0
glucose_concentration       0
diastolic_blood_pressure    0
triceps_sf_thickness        0
serum_insulin               0
bmi                         0
d_pedigree_function         0
years_of_age                0
diabetes                    0
dtype: int64
```

```
Correlation Matrix:
                         times_pregnant  ...  diabetes
times_pregnant                1.000000  ...  0.221898
glucose_concentration         0.128213  ...  0.492782
diastolic_blood_pressure      0.208615  ...  0.165723
triceps_sf_thickness          0.081770  ...  0.214873
serum_insulin                 0.025047  ...  0.203790
bmi                           0.021559  ...  0.312038
d_pedigree_function          -0.033523  ...  0.173844
years_of_age                  0.544341  ...  0.238356
diabetes                      0.221898  ...  1.000000

[9 rows x 9 columns]

Sample Used for Distance:
   times_pregnant  glucose_concentration  ...  years_of_age  diabetes
0               6                  148.0  ...            50         1
1               1                   85.0  ...            31         0

[2 rows x 9 columns]

Euclidean Distance:
[[ 0.         66.91095707]
 [66.91095707  0.        ]]

Manhattan Distance:
[[  0.    107.276]
 [107.276   0.   ]]

Cosine Distance:
[[0.         0.03163284]
 [0.03163284 0.        ]]

Standardized Data:
   times_pregnant  glucose_concentration  ...  years_of_age  diabetes
0        0.639947               0.866045  ...      1.425995  1.365896
1       -0.844885              -1.205066  ...     -0.190672 -0.732120
2        1.233880               2.016662  ...     -0.105584  1.365896
3       -0.844885              -1.073567  ...     -1.041549 -0.732120
4       -1.141852               0.504422  ...     -0.020496  1.365896

[5 rows x 9 columns]
```

```
Euclidean Distance After Scaling:
[[0.         3.99437954]
 [3.99437954 0.        ]]

Manhattan Distance After Scaling:
[[ 0.         10.30227071]
 [10.30227071  0.        ]]

Cosine Distance After Scaling:
[[0.         1.65734516]
 [1.65734516 0.        ]]
PS D:\swayam>
```




## Conclusion

Correlation analysis, distance measures, and data preprocessing techniques were implemented on the Pima Indians Diabetes Dataset. Correlation analysis helped identify relationships between medical variables, while distance measures quantified the similarity or dissimilarity between observations. Data preprocessing techniques helped improve the consistency and suitability of the data for further analysis and machine learning applications.

Questions to be Answered by Students :

**1. Which pair of variables has the strongest positive correlation in the dataset?**

The strongest positive correlation is typically between Glucose and Outcome (Diabetes), with a correlation coefficient of approximately 0.49. This indicates that higher glucose levels are associated with a greater likelihood of diabetes.

**2. Which pair of variables has the strongest negative correlation, if any?**

The Pima Indians Diabetes Dataset generally does not contain any strong negative correlations. Most variables have weak positive correlations, and any negative correlations are very small (close to 0), indicating little or no inverse linear relationship.

**3. How do the Euclidean, Manhattan, and Cosine distances differ for the selected observations?**

- **Euclidean Distance:** Measures the straight-line distance between two observations.
- **Manhattan Distance:** Measures the sum of the absolute differences between corresponding features.
- **Cosine Distance:** Measures the difference in the direction (angle) of two feature vectors rather than their magnitude.

Thus, Euclidean and Manhattan distances depend on the actual values, while Cosine distance focuses on the similarity in the pattern of the observations.

**4. Which preprocessing technique was applied to the invalid or missing values, and why was it selected?**

Median imputation was used to replace invalid zero values (treated as missing values) in medical attributes such as Glucose, BloodPressure, SkinThickness, Insulin, and BMI. The median was chosen because it is less affected by outliers and provides a more reliable estimate for medical data.

**5. How does normalization or standardization affect the calculated distance between observations?**

Normalization and standardization scale all features to a comparable range, preventing variables with larger numerical values from dominating the distance calculations. As a result, Euclidean and Manhattan distances become more balanced, leading to more accurate similarity measurements and improving the performance of distance-based machine learning algorithms.